

DAY 12

TWO POINTER TECHNIQUE

A practical guide from intuition → pattern recognition → C++ → interview problems

Save this. Practice it. Then teach it.

01 • What is Two Pointer?

Use two indices to process a sequence efficiently.

Instead of repeatedly scanning the same array, maintain two positions—often LEFT and RIGHT.

Common setups:

- One pointer at each end
- Two pointers moving in the same direction
- Fast & slow pointers

Core idea: move pointers based on what the current state tells you.

02 • Why do we need it?

Turn many $O(n^2)$ brute-force scans into $O(n)$.

Typical brute force:

Check every pair $\rightarrow O(n^2)$

Two pointers can eliminate unnecessary comparisons by exploiting structure such as:

- Sorted order
- A monotonic condition
- A fixed/variable window
- Relative pointer speed

Interview payoff: faster solutions + simpler reasoning.

03 • The Mental Model

Think: “What does each pointer represent?”

Before coding, define:

1. What does LEFT represent?
2. What does RIGHT represent?
3. When can LEFT move safely?
4. When can RIGHT move safely?
5. What condition ends the loop?

If you cannot answer these, don't code yet.

04 • Pattern A: Opposite Ends

Most classic two-pointer pattern.

Start: $\text{left} = 0$, $\text{right} = n - 1$

While $\text{left} < \text{right}$:

- Evaluate $\text{arr}[\text{left}]$ and $\text{arr}[\text{right}]$
- If condition says left must increase $\rightarrow \text{left}++$
- If condition says right must decrease $\rightarrow \text{right}--$

Best for sorted arrays and “find a pair” problems.

05 • Example: Two Sum II

Sorted array + target $\rightarrow$ two pointers.

Array: [2, 7, 11, 15], target = 9

left=2, right=15 $\rightarrow$ sum 17 $\rightarrow$ too large $\rightarrow$ right--

left=2, right=11 $\rightarrow$ sum 13 $\rightarrow$ too large $\rightarrow$ right--

left=2, right=7 $\rightarrow$ sum 9 $\rightarrow$ FOUND!

The sorted order tells us which pointer can move safely.

06 • C++ Implementation

Clean interview-ready template.

```
vector<int> twoSum(vector<int>& a, int target) {  
    int left = 0, right = a.size() - 1;  
  
    while (left < right) {  
        int sum = a[left] + a[right];  
  
        if (sum == target) return {left, right};  
        if (sum < target) left++;  
        else right--;  
    }  
    return {};  
}
```

07 • Pattern B: Fast & Slow

Two pointers, different speeds.

slow usually moves one step

fast usually moves two steps

Common uses:

- Detect cycle in linked list
- Find middle of linked list
- Remove duplicates / in-place processing

Important: “Two pointers” does NOT always mean left/right.

08 • Pattern C: Same Direction

Pointers can build or maintain a valid region.

Example idea: remove duplicates from a sorted array.

slow = position of last accepted element

fast = scanner

As fast explores the array, slow advances only when a new valid value is found.

This is powerful for in-place array transformations.

09 • Complexity

Why interviewers love the technique.

Typical two-pointer scan:

Time: $O(n)$

Space: $O(1)$

Why $O(n)$? Each pointer usually moves forward/backward a limited number of times.

Caution: if you nest a full two-pointer scan inside another loop, total complexity may become $O(n^2)$. Always analyze the whole algorithm.

10 • LeetCode Practice

Basic → intermediate → pattern recognition.

1. #167 Two Sum II – Input Array Is Sorted

Learn: opposite-end pointers

2. #125 Valid Palindrome

Learn: comparing from both ends

3. #11 Container With Most Water

Learn: greedy pointer movement + proof intuition

Goal: don't memorize code. Explain WHY a pointer moves.

11 • Real-World Analogy

Finding two seats that fit a group.

Imagine seats sorted by capacity.

Start with smallest + largest capacity.

If the pair is too large → reduce the larger side.

If too small → increase the smaller side.

You avoid checking every possible pair because the ordering gives you information.

12 • Common Mistakes

The bugs that cost interviews.

- ✗ Moving both pointers without justification
- ✗ Using two pointers on an unsorted array without a valid reason
- ✗ Wrong loop condition: $\text{left} < \text{right}$ vs $\text{left} \leq \text{right}$
- ✗ Forgetting duplicate handling
- ✗ Coding before defining pointer invariants

Rule: every pointer movement needs a reason.

13 • Interview Tip

Don't just say “this is $O(n)$.” Prove it.

Strong interview explanation:

“I use two pointers because the input is sorted. If the sum is too small, moving RIGHT left would only make it smaller, so LEFT is the only safe move. If the sum is too large, RIGHT is the safe move. Each pointer moves at most n times, so time is $O(n)$ and extra space is $O(1)$.”

That shows understanding—not memorization.

14 • Quick Challenge

Pause before revealing the idea.

Given sorted array:

[1, 3, 4, 6, 8, 10, 13]

Target = 14

Question:

Which pointers move at each step?

Try to solve it on paper first.

Answer: start $1 + 13 = 14 \rightarrow$ found immediately.

15 • What I'll Learn Tomorrow

DAY 13 → Sliding Window

Two Pointer is the foundation.

Sliding Window builds on the same idea to maintain a contiguous range efficiently.

Next:

-  What is a window?
-  Fixed vs variable window
-  C++ templates
-  Examples
-  Complexity
-  LeetCode practice

GitHub: https://github.com/ritesh2687/dsa_ppt_post